Universidade Federal de Minas Gerais

ELT127 -Automação em Tempo Real

Relatório Técnico – TP

Etapa 1 – Arquitetura, Processos e Sincronização

Gerenciamento de Frota Robótica para Logística

Lucca Rigueira Sant'ana Nogueira
Matheus Muniz Soares
Pedro Henrique Chalfun de Aquino

Prof. Leandro Freitas

Belo Horizonte, 19 de outubro de 2026

Sumário

1 Introdução e objetivos da etapa 2

2 Arquitetura do sistema 2

3 Implementação dos processos 3

3.1 Processo principal e coordenação sequencial 3

3.2 Criação dinâmica dos processos dos robôs 3

3.3 O que permanece isolado entre processos 3

4 Implementação das threads 3

4.1 Thread de comando de navegação 3

4.2 Thread de status e tratamento de sensores 4

4.3 Thread do coletor de dados 4

5 Comunicação entre processos e troca de dados 4

5.1 Canal IPC e enquadramento das mensagens 4

5.2 Sequência de uma operação de transporte 4

6 Mecanismos de sincronização 5

6.1 Mutexes: exclusão mútua dentro do robô 5

6.2 Variáveis de condição: espera sem sondagem ocupada . . . 5

6.3 Exclusão mútua no canal de envio 6

6.4 Sincronização lógica de equipes e de passagens 6

7 Persistência e rastreabilidade 6

8 Testes e evidências de troca de dados 7

9 Resultados da etapa e limitações 7

10 Compilação e reprodução 7

11 Conclusão 8

1 Introdução e objetivos da etapa

Nesta primeira etapa, desenvolvemos a arquitetura do sistema de gerenciamento de frota robótica e implementamos seus principais componentes. A solução contempla processos independentes para os robôs, comunicação entre processos, threads de operação e mecanismos de sincronização. Também implementamos recursos de persistência e realizamos testes de integração para verificar o funcionamento da arquitetura. As funcionalidades previstas para etapas posteriores permanecem fora do escopo desta entrega.

2 Arquitetura do sistema

Organizamos a arquitetura em três partes: coordenação logística, execução dos robôs e persistência de dados em SQLite. O processo central gerencia missões e recursos simulados, enquanto cada robô executa em um processo independente.

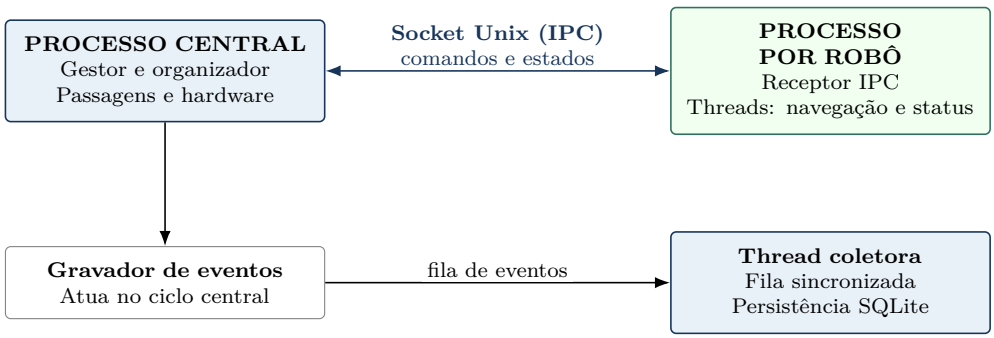


Figura 1: Arquitetura da primeira etapa

A Tabela 1 resume as responsabilidades dos componentes.

Tabela 1: Responsabilidades dos componentes implementados.

Módulo	Contexto	Responsabilidade
`AplicacaoCentral`	Processo central; laço principal	Processa mensagens, aciona logística, hardware e passagens.
`GestorRobos`	Mesmo processo central	Cria/encerra processos, acompanha PIDs, gerações, mensagens e prontidão.
`AgenteRobo`	Processo individual	Compartilha estado local entre threads de navegação e status.
`OrganizadorLogistica`	Ciclo central	Reserva equipes, define missões, acompanha fases e retomadas.
`ControlePassagens`	Ciclo central	Mantém fila, capacidade, ocupação e permissões de tráfego.
`AdaptadorHardwareLocal`	Ciclo central	Converte setpoints válidos em novas leituras de sensores.
`ColetorDados`	Thread própria	Consome eventos da fila e persiste registros no SQLite.

3 Implementação dos processos

3.1 Processo principal e coordenação sequencial

O executável `simulador_exec` permite selecionar cenários, banco de dados e modo interativo. A função principal inicia a aplicação, executa seus ciclos e coordena o encerramento. No modo automático, cada cenário possui limite de execução de 120 segundos.

O ciclo principal recebe mensagens IPC, atualiza sensores e permissões e processa as missões após eventos ou, em períodos de inatividade, a cada 200 ms aproximadamente. Os componentes de logística, passagens e hardware são executados sequencialmente. Com isso, concentramos o acesso ao estado central em um único fluxo de execução, reduzindo a necessidade de bloqueios.

3.2 Criação dinâmica dos processos dos robôs

Ao adicionar um robô, o gestor valida seu identificador, cria um par de sockets Unix e inicia um novo processo com `posix_spawn()`. O processo central mantém o PID, a identificação lógica, a geração e o canal de comunicação de cada agente.

A inicialização compreende a abertura do canal, a criação do processo e a confirmação de prontidão. O gestor só envia tarefas após receber `RoboPronto`. O número de geração permite distinguir reinicializações de um mesmo robô e descartar mensagens de instâncias anteriores.

No encerramento, o gestor solicita a parada e monitora o processo com `waitpid()`. Caso necessário, envia `SIGTERM` após dois segundos e `SIGKILL` após três. Desconexões inesperadas tornam o robô indisponível, sem reinicialização automática.

3.3 O que permanece isolado entre processos

Não há memória compartilhada entre o robô e o gestor. As informações são transmitidas por mensagens serializadas, cuja consistência depende da validação de geração, sequência, confirmações e leituras de sensores.

4 Implementação das threads

4.1 Thread de comando de navegação

Cada robô possui uma thread de navegação com período nominal de **100 ms**. Ela verifica a missão, o nível de bateria, possíveis falhas e confirmações pendentes. Havendo um

próximo destino, solicita acesso à passagem antes de enviar o comando de movimento. A rota só avança quando o sensor confirma a posição esperada.

Ao concluir o percurso, o agente passa à fase correspondente da missão, como acoplamento, descarga ou recarga. Esse procedimento evita divergências entre posição estimada e posição confirmada.

4.2 Thread de status e tratamento de sensores

A thread de status possui período nominal de **500 ms**, com ativação antecipada diante de alterações de estado ou novos sensores. Ela processa leituras, acumula a distância percorrida, atualiza a disponibilidade e publica o estado do robô.

Os registros incluem identidade, missão, fase, posição, bateria, falhas e distância. Leituras antigas ou incompatíveis com a missão são descartadas; leituras intermediárias são consideradas no cálculo do deslocamento.

4.3 Thread do coletor de dados

O coletor utiliza uma thread independente para retirar eventos de uma fila sincronizada e gravá-los no SQLite. Dessa forma, as operações logísticas não precisam aguardar cada escrita em disco.

No encerramento, o coletor processa os eventos pendentes antes de fechar o banco. Erros de persistência são registrados e podem ser encaminhados ao processo central.

5 Comunicação entre processos e troca de dados

5.1 Canal IPC e enquadramento das mensagens

Os processos se comunicam por sockets Unix do tipo `SOCK_STREAM`. Como o fluxo não delimita mensagens, cada quadro contém um prefixo de quatro bytes que informa seu tamanho. O receptor recompõe mensagens fragmentadas, respeitando o limite de 64 KiB.

O protocolo transmite identificação, geração, sequência, comandos, missões e dados operacionais. A decodificação valida formato, versão e limites dos campos. O envio utiliza `poll()`, com prazo de até 1 s; o gestor aplica limites de 2 s para prontidão e 3 s para comunicação ou confirmação.

5.2 Sequência de uma operação de transporte

A Figura 2 apresenta a sequência de autorização, movimento e confirmação por sensor.

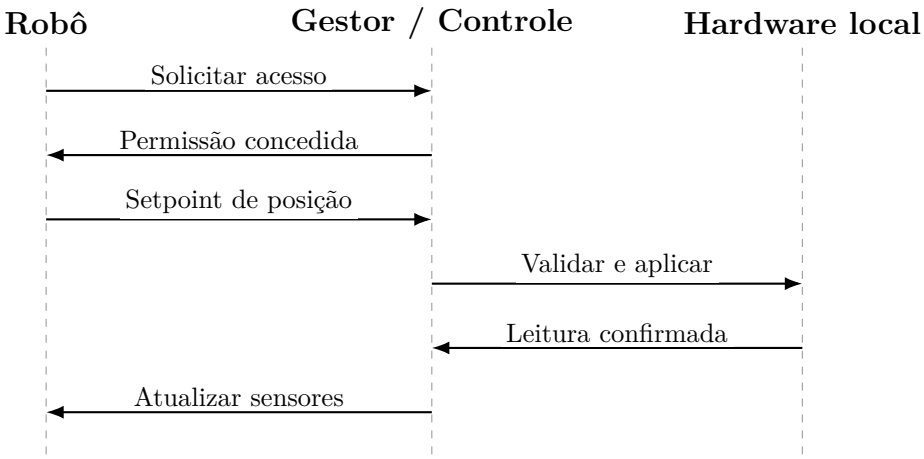


Figura 2: Fluxo de confirmação de um passo de navegação.

Na atribuição de missões, cada comando recebe uma confirmação associada ao seu identificador. Também são verificados geração, sequência e revisão da missão, evitando a aceitação de informações desatualizadas.

6 Mecanismos de sincronização

6.1 Mutexes: exclusão mútua dentro do robô

Utilizamos `std::mutex` para proteger o estado, a rota e as leituras de sensores compartilhadas entre as threads e o receptor de mensagens. Isso impede alterações simultâneas que poderiam comprometer a consistência da navegação.

O controle dos bloqueios utiliza `std::lock_guard` e `std::unique_lock`. Mutexes específicos protegem o início e a parada das threads e a sequência de envio das mensagens IPC.

6.2 Variáveis de condição: espera sem sondagem ocupada

As threads utilizam `std::condition_variable` para aguardar comandos, leituras ou encerramento sem consultar o estado continuamente. Predicados verificam a condição de retomada e evitam efeitos de despertares espúrios.

No coletor, a variável de condição sinaliza novos eventos ou encerramento. O mutex é liberado antes das operações SQLite, permitindo que outras threads continuem adicionando eventos à fila.

6.3 Exclusão mútua no canal de envio

O canal IPC utiliza `std::timed_mutex` para impedir que bytes de mensagens diferentes sejam intercalados no mesmo socket. A proteção do envio é independente do mutex utilizado para o estado interno do robô.

6.4 Sincronização lógica de equipes e de passagens

O transporte em equipe emprega uma barreira lógica: a missão só prossegue quando todos os integrantes confirmam o acoplamento. O movimento coletivo exige propostas compatíveis de posição e destino, e a descarga depende do desacoplamento completo.

As passagens possuem controle de capacidade, sentido e ocupação. Equipes acopladas reservam vagas em conjunto; deslocamentos opostos aguardam a liberação do corredor. A saída é confirmada por sensores antes da liberação das vagas.

Como essas estruturas são manipuladas pelo ciclo central, o controle ocorre por filas e permissões, sem uso de semáforos POSIX.

Tabela 2: Mapa de sincronização: recurso, concorrência e proteção.

Recurso	**Quem acessa**	**Proteção existente**
Estado, rota e sensores do robô	Laço receptor + 2 threads	`mutex_`, `cv_`, validações de versão e comando.
Ciclo de vida das threads	Inicialização e parada do agente	`ciclo_vida_`, `notify_all()`, `join()`.
Envio no socket do robô	Threads emissoras	`send_mutex` e `timed_mutex` do canal.
Fila de persistência	Produtoras + thread coletora	`mutex_fila`, `condicao_fila`, limite de fila.
Estado da frota, missão e corredor	Apenas ciclo central	Propriedade exclusiva do estado e validação por mensagens.
Posição da equipe acoplada	Propostas dos robôs	Barreira de confirmação por integrantes, missão, passo e destino.

7 Persistência e rastreabilidade

Os eventos operacionais são encaminhados ao coletor e armazenados em quatro tabelas SQLite: `runs`, `robot_events`, `cargo_transports` e `cargo_members`. Os registros permitem relacionar execuções, robôs, missões, cargas e integrantes das equipes.

Cada evento é gravado em transação, com reversão em caso de falha. Restrições de identidade evitam duplicações e os registros utilizam horários Unix em milissegundos. Os prazos operacionais são medidos separadamente por relógio monotônico.

Também implementamos um buffer circular de eventos, avaliado em teste específico. No fluxo principal, porém, os eventos são enviados diretamente ao coletor.

8 Testes e evidências de troca de dados

Implementamos para o projeto uma suíte de testes automatizados utilizando GoogleTests, que podem ser compilados e executados pelo terminal, garantindo o correto funcionamento do código. Esses teste, cobrem a comunicação e a gravação persistente dos dados, utilizando o SQLite, utilizando os mecanismos de exclusão mútua para garantir o bom funcionamente em situações com concorrência.

Além disso, para alguns casos específicos, implementamos testes manuais de forma pontual, como por exemplo, alguns testes relacionados ao DataLogger e ao banco de dados.

9 Resultados da etapa e limitações

Concluímos a primeira etapa com os principais componentes implementados e testes funcionais de processos, threads, comunicação IPC, sincronização e persistência. A separação de responsabilidades reduziu o compartilhamento de memória e concentrou os bloqueios nas operações realmente concorrentes.

Para as próximas etapas, permanecem a interface gráfica, a integração MQTT e as avaliações de latência, variação dos períodos das threads e escalabilidade. Os períodos de 100 e 500 ms são nominais, pois ainda não realizamos medições de pior caso nem configuramos escalonamento de tempo real estrito.

10 Compilação e reprodução

A compilação e a execução dos cenários podem ser reproduzidas, a partir da raiz do projeto, pelos comandos:

```
cmake -S . -B build -DBUILD_TESTING=OFF
cmake --build build --parallel 2
./build/app/simulador_exec --cenario base
./build/app/simulador_exec --cenario disponibilidade
./build/app/simulador_exec --cenario processos
```

Para definir outro arquivo SQLite, utiliza-se `-banco logs/nome-da-execucao.db`. Com GoogleTest disponível, a suíte pode ser executada por:

```
cmake -S . -B build -DBUILD_TESTING=ON
cmake --build build --parallel 2
ctest --test-dir build --output-on-failure
```

Consulta SQL para verificar transportes e integrantes:

```
SELECT t.mission_id, t.cargo_id, t.status,
       m.robot_id
FROM cargo_transports AS t
JOIN cargo_members AS m
  ON m.run_id = t.run_id
 AND m.mission_id = t.mission_id
ORDER BY t.mission_id, m.robot_id;
```

11 Conclusão

Nesta primeira etapa, definimos e implementamos a arquitetura básica do sistema de gerenciamento de frota robótica. A organização em processos independentes, o uso de sockets Unix e a aplicação de mutexes e variáveis de condição permitiram coordenar os agentes e proteger os dados compartilhados. O controle sequencial do gestor e a fila de persistência contribuíram para separar as responsabilidades do sistema.

Os testes realizados confirmaram a transmissão e validação de mensagens IPC, a persistência de 200 eventos produzidos concorrentemente e o funcionamento dos três cenários de integração avaliados. Assim, consideramos concluídos os objetivos previstos para a Etapa 1, com uma base funcional para a continuidade do projeto. A integração de novas interfaces e as avaliações temporais permanecem previstas para as etapas seguintes.